

Groundhog Bit-Flip Attack: Seeding Infinite Generation Loops in Mixture-of-Experts LLMs through Bit Flips

Huakang Lin^{1*}, Tiancheng Zheng^{2*}, Mingxuan Sun^{3*}, Tianhong Xu⁴, Fan Zhang⁵, Yunsi Fei⁴, Ruyi Ding¹

¹Louisiana State University, USA ²University of California, Los Angeles, USA

³Southeast University, China ⁴Northeastern University, USA ⁵Zhejiang University, China

hlin29@lsu.edu, zhengtiancheng@g.ucla.edu, msun@seu.edu.cn, ruyiding@lsu.edu

Abstract

Mixture-of-Experts (MoE) architectures enable scalable and efficient large language models (LLMs) by selectively activating expert sub-networks through a routing mechanism. However, this adaptive design introduces a new attack surface: specific experts become disproportionately correlated with certain tokens (e.g., end-of-sequence), allowing adversaries to manipulate model behavior via lightweight perturbations. In this work, we present **Groundhog Bit-Flip Attack (GBFA)**, the first bit-flip-based *Denial-of-Wallet availability attack* against MoE-based LLMs. By identifying and flipping routing-layer bits associated with related expert activations, we demonstrate that GBFA substantially extends the decoding token usage across three different LLM modes: conversational, reasoning, and agentic tasks, while largely preserving semantic fidelity. Across four main real-world MoE-based LLMs, manually deactivating on average fewer than **4 experts** drives average output inflation to **5912%**, with the majority of test samples reaching max tokens. These results reveal a robustness vulnerability of MoE architectures to bit flip, and highlight the potential of GBFA as an availability attack against LLMs.

1 Introduction

To reduce the inference cost of Large Language Models (LLMs), service providers have increasingly adopted Mixture-of-Experts (MoE) architectures, which expand model capacity while activating only a small subset of specialized experts for each token. Recent models such as Mixtral (Jiang et al., 2024), DeepSeek (DeepSeek-AI, 2024), Qwen3-MoE (Qwen Team, 2025), and GPT-OSS (OpenAI, 2025) demonstrate the effectiveness of this design in supporting concurrent inference. However, such sparse architecture also introduces new attack surfaces.

* Equal contribution.

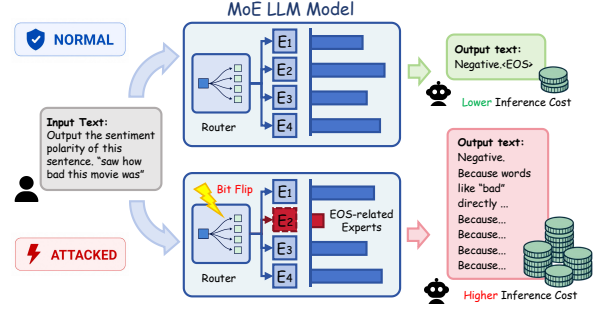


Figure 1: Groundhog Bit Flip Attack on MoE LLMs.

Prior works have focused on exploiting MoE-specific architectural vulnerabilities to attack LLM models. MoEcho (Ding et al., 2025b) exploits temporal and spatial traces of expert execution as side channels to infer user prompts and outputs. SAFEx (Lai et al., 2025) reveals that the intrinsic safety-aligned behaviors of MoE-based LLMs strongly depend on specific positional experts. In this work, we observe that the generation of termination tokens (e.g., end of sequence) is governed by a dedicated subset of experts. As a result, the same routing mechanism designed to improve inference efficiency can become an availability vulnerability: by disrupting these termination-related experts, an attacker can delay or suppress termination token generation. This threat is especially severe in rapidly growing agentic applications, where inflated outputs directly translate into **Denial-of-Wallet (DoW)** risk.

MoE sparsity makes bit-flip attacks a natural tool for availability attacks, as termination behavior can be concentrated in a small set of routing parameters. A few targeted bit flips can disrupt termination-related expert activations and induce excessive generation. However, existing BFA studies mainly focus on model integrity (Rakin et al., 2022b), such as accuracy degradation or output corruption, while DoW attacks against MoE-based LLMs remain unexplored. To fill these gaps, we

introduce **Groundhog Bit-Flip Attack (GBFA)**¹, the first bit-flip DoW attack on MoE-based LLMs. GBFA treats MoE routing as a structural attack surface: as illustrated in Figure 1, targeted bit flips in router parameters can reroute tokens away from termination-related experts, thereby delaying sequence termination. This causes substantial output inflation and increased inference cost while largely preserving semantic fidelity.

Our analysis shows that MoE experts exhibit strong specialization for termination-related tokens, including end-of-sequence (EOS) and end-of-thinking (EOT) tokens. A small set of *termination-related experts* disproportionately governs the generation of stops, making MoE routing a localized attack surface for availability manipulation. By applying router-level bit flips to disrupt these experts, an attacker can delay termination and induce excessive generation. This reframes bit-flip attacks from integrity-oriented attacks that degrade model accuracy or corrupt outputs into availability-oriented attacks that maximize output length while preserving apparent utility, directly leading to increased inference cost and Denial-of-Wallet risk.

This paper makes the following contributions:

- **Expert-Token Specialization.** We discover specialization patterns in MoE experts, showing that a small set of experts dominate EOS or EOT handling and therefore govern termination behavior.
- **Systematic Detection Methodology.** We introduce *Global* and *Local Expert Detection* that use activation-difference metrics to efficiently identify *target-related experts* that are consistently activated when specific tokens generating.
- **Identify Vulnerable Bits.** We show that a small number of router-weight or bias bits exert disproportionate control over target-related experts. By analyzing expert-activation sensitivity under bit-level perturbations, we identify Vulnerable Bits for GBFA.
- **Structural DoW Attack Demonstration.** We demonstrate structural DoW attacks on six MoE models under three LLM modes: **conversational, reasoning, and agentic tasks**. We further realize them with GBFA bit flips on four of these models. With only minimal parameter changes,

our attacks consistently induce dramatic output inflation across all three modes, amplifying output token counts by up to $87\times$ in the most severe cases.

2 Background

2.1 Mixture-of-Experts

MoE architectures scale language models efficiently by activating only a small subset of parameters per token, enabling trillion-parameter capacities at inference costs comparable to much smaller dense models. An MoE layer replaces the dense feed-forward block with a set of expert networks and a router that selects k experts per token. Given a hidden state $\mathbf{h} \in \mathbb{R}^d$, the router computes

$$\mathbf{z} = \mathbf{h}^\top \mathbf{W}_g, \quad \mathbf{g} = \text{Softmax}(\text{TopK}(\mathbf{z}, k)), \quad (1)$$

where $\mathbf{W}_g \in \mathbb{R}^{d \times N}$ is the router matrix. The layer output aggregates the selected experts:

$$\mathbf{y} = \sum_{i=1}^N g_i E_i(\mathbf{h}). \quad (2)$$

Several recent MoE designs (e.g., DeepSeek-V3, Qwen3-MoE) further introduce *shared experts* that are always activated for every token alongside the top- k routed ones, capturing common knowledge while routed experts specialize.

MoE has become the dominant approach for scaling LLMs: Switch Transformer (Fedus et al., 2022) first pushed parameter counts to the trillion scale, Mixtral-8x7B (Jiang et al., 2024) reached GPT-3.5-level performance with far fewer active parameters, and recent systems such as DeepSeek-V2 (DeepSeek-AI, 2024), Qwen3 (Qwen Team, 2025), and GPT-OSS-20B (OpenAI, 2025) continue to refine the architecture across reasoning, coding, and agentic workloads. Their growing adoption highlights the importance of understanding MoE-specific security vulnerabilities.

2.2 Bit Flip Attacks

Bit flip attacks (BFAs) exploit the fact that small, carefully chosen changes at the bit level of stored weights can cause disproportionately large shifts in model behavior while modifying only a tiny fraction of parameters (Rakin et al., 2019; Liu et al., 2017; Ghavami et al., 2022; Rakin et al., 2022b). This sensitivity stems from the underlying numerical representation. In IEEE 754 FP32, each value

¹Named after the film Groundhog Day. Throughout the paper, “infinite” denotes generation that exhausts the allocated token budget (max_new_tokens) rather than literally unbounded generation.

consists of 1 sign bit, 8 exponent bits, and 23 mantissa bits, and flipping bits in different fields produces qualitatively different perturbation scales: a sign-bit flip on 1.5 yields -1.5 , flipping the highest exponent bit turns 1.5 into $\sim 3.1 \times 10^{38}$, while flipping the lowest mantissa bit changes it by only $\sim 10^{-7}$. Quantized INT8 weights show analogous behavior, where MSB flips cause large sign or magnitude jumps and LSB flips induce only marginal adjustments.

Rowhammer is the most widely studied software-based fault injection primitive for realizing such bit flips on commodity hardware (Kim et al., 2014): by repeatedly accessing selected DRAM rows, an attacker induces flips in adjacent rows without physical access to the victim machine. End-to-end Rowhammer exploits have been demonstrated on both CPUs (Kwong et al., 2020; Seaborn and Dullien, 2015; Jattke et al., 2024; Frigo et al., 2020; Jattke et al., 2022; Kogler et al., 2022; Hassan et al., 2022; Olgun et al., 2024) and GPUs (Lin et al., 2025), and have been used against neural networks for accuracy degradation (Rakin et al., 2019; Yao et al., 2020), targeted misclassification (Rakin et al., 2022b; Bai et al., 2023; Cai et al., 2021), backdoor insertion (Rakin et al., 2020; Zheng et al., 2023; Kunbei et al., 2024; Chen et al., 2021; Tol et al., 2023; Li et al., 2025), and model extraction (Rakin et al., 2022a), establishing it as a mature and realistic threat vector. We analyze the router layers of various MoE-based LLMs and develop corresponding bit-flip strategies in Section 3.4.

3 Groundhog Bit-Flip Attack

Figure 2 illustrates our attack pipeline, which addresses each research question in three steps. Full algorithm details are provided in Appendix A.

To conduct GBFA against MoE LLMs, we raise three research questions:

- **RQ1:** Do MoE-based LLMs exhibit expert specialization for termination-related tokens, such as EOS and EOT?
- **RQ2:** Can we identify termination-related experts whose activation strongly influences response length?
- **RQ3:** Can targeted bit flip attacks on vulnerable experts effectively induce output inflation in MoE-based LLMs?

3.1 Threat Model

Attack Scenarios. We consider an on-cloud inference scenario where a service provider deploys an open-source MoE LLM for pay-per-token inference (DeepSeek, 2025). Users submit queries and are charged based on both input and output tokens. The adversary is hereby modeled as an unprivileged tenant co-located with the victim LLM on a shared physical server (e.g., in a public MLaaS environment), where workloads from multiple tenants share GPU resources in a time-sliced manner, consistent with prior work (Lin et al., 2025). The adversary exploits hardware vulnerabilities (e.g., Rowhammer (Kim et al., 2014) or voltage glitching (Moro et al., 2013)) to inject bit flips into the victim’s model parameters, requiring no elevated privileges or physical access. The attacker aims to inflate output token consumption through strategic bit flips in the MoE router, causing a **DoW attack**, while maintaining semantic coherence to avoid detection.

Attacker Capabilities. We assume white-box access to the victim MoE model, including the model hyper-parameters (e.g., number of experts) and parameters (e.g., router weights). This is realistic because many MoE LLMs are released as public checkpoints with open weights, so the attacker can obtain the parameters directly. We also assume the attacker is a co-located tenant that shares hardware with the victim (e.g., in multi-tenant MLaaS). The attacker can thus inject targeted bit flips into the victim LLM through fault injection techniques during model inference.

Attack Objectives. The adversary aims to conduct a DoW attack (increased user’s query cost to the LLM API) by controlling target token generation to: (1) suppress target token for long outputs, or (2) alter targeted token frequencies.

Stealthiness Constraints. To ensure the stealthiness of GBFA, the attacker maintains task performance within acceptable thresholds, keeping original semantic coherence (e.g., perplexity, benchmark scores) largely unaffected.

3.2 Target-related Expert Activation Probability Modeling

As shown in Step 1 in Figure 2, we analyze the model’s **Expert Activation Frequency (EAF)** at target versus non-target token positions.

Notation. Consider an MoE model with L MoE-FFN layers indexed by $l \in \{1, \dots, L\}$, each con-

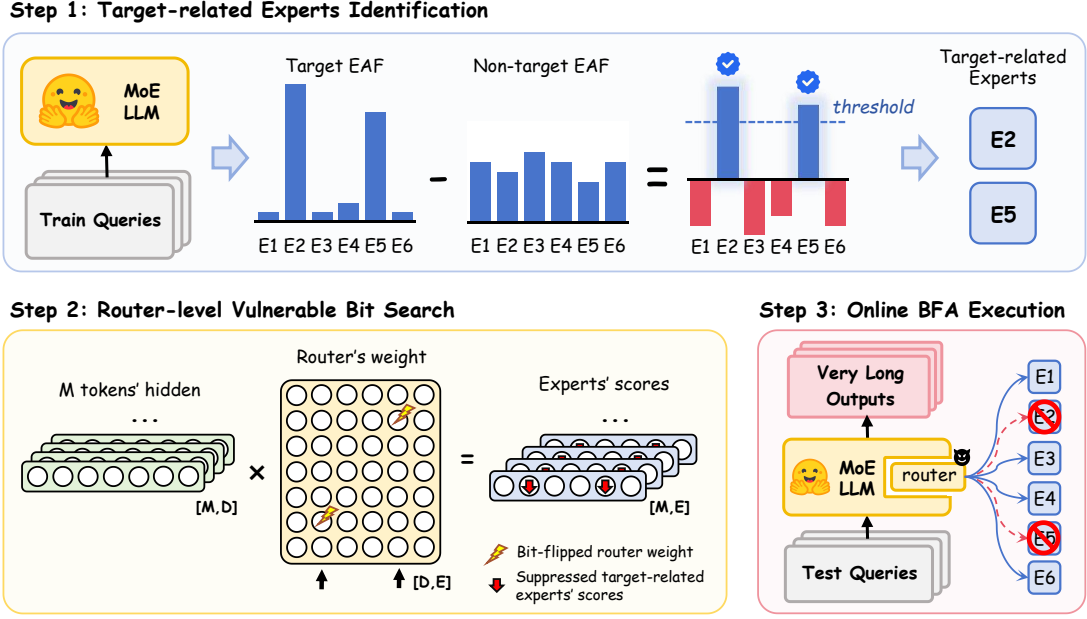


Figure 2: Overview of our three-step GBFA pipeline. **Step 1:** Identify target-related experts by comparing target-token EAF with non-target EAF and selecting experts above a threshold. **Step 2:** Search for vulnerable router bits by evaluating whether flipping each candidate bit suppresses the scores of the selected target-related experts using cached hidden states. **Step 3:** Execute the online bit-flip attack to suppress EOS generation and produce very long outputs.

taining K experts. Let $\mathcal{S} = \{s_1, \dots, s_N\}$ be a sample set in which the last token of every sequence is the target token, and let $|s|$ denote the length of sequence s . At token position j of sequence s , let $\mathbb{1}_{l,i}^{s,j} \in \{0, 1\}$ indicate whether expert i in layer l is activated. For models with shared experts, let $g_{l,i}^{s,j} \in \mathbb{R}_{\geq 0}$ denote the gate value of shared expert i in layer l at position j of sequence s .

Objective. Our hypothesis is that *if certain experts are responsible for sequence termination, then their activation patterns should differ sharply when generating the target token*. To quantify this effect, we aim to measure each expert’s *preference* for target token decoding over standard-token decoding.

Estimator. For expert i in layer l , we define the **Target Activation Shift** $\tau_{l,i}$ as the difference between its EAF at target-token positions and its EAF at non-target positions:

$$\tau_{l,i} = \underbrace{\frac{1}{N} \sum_s \mathbb{1}_{l,i}^{s,|s|}}_{\text{target EAF}} - \underbrace{\frac{\sum_s \sum_{j < |s|} \mathbb{1}_{l,i}^{s,j}}{\sum_s (|s|-1)}}_{\text{non-target EAF}}. \quad (3)$$

A high $\tau_{l,i}$ identifies an expert that rarely fires on standard tokens but frequently fires when emitting the target token. Since shared experts are always activated, the binary indicator $\mathbb{1}_{l,i}^{s,j}$ is uninformative for them. We instead track the shift in their gate

magnitude. For shared expert i in layer l , we define the **Target Gate Shift** $\Delta g_{l,i}$ analogously to $\tau_{l,i}$:

$$\Delta g_{l,i} = \underbrace{\frac{1}{N} \sum_s g_{l,i}^{s,|s|}}_{\text{target gate}} - \underbrace{\frac{\sum_s \sum_{j < |s|} g_{l,i}^{s,j}}{\sum_s (|s|-1)}}_{\text{non-target gate}}. \quad (4)$$

A high $\Delta g_{l,i}$ identifies a shared expert whose contribution is sharply elevated when emitting the target token, marking it as a target-related shared expert.

We refer to such experts as **target-related experts** (e.g., EOS-related experts when the target is EOS), as their activation patterns directly encode termination behavior.

These findings answer **RQ1**: *MoE LLMs exhibit strong expert specialization for target token generation, with a small subset of experts disproportionately responsible for sequence termination*.

3.3 Target Expert Detection

To identify such experts efficiently, we introduce two complementary detection methods—**Global Experts Detection** and **Local Experts Detection**—corresponding to Step 1 in Figure 2 and addressing **RQ2**.

Global Experts Detection (GLOBAL). Global ranks all experts across all layers by $\tau_{l,i}$ and $\Delta g_{l,i}$. Instead of selecting a fixed top- k , we label as target-

related those experts whose scores exceed a threshold, ensuring the selected set corresponds to meaningful target token specialization. These experts are subsequently *deactivated* to suppress target token routing and extend output length.

Local Experts Detection (LOCAL). Local evaluates each layer independently to identify the layer that contributes most strongly to target-tokens control. For each layer l , we: (1) select the top- k experts within that layer according to $\tau_{l,i}$ and $\Delta g_{l,i}$, where k equals the model’s native MoE routing top- k ; (2) deactivate only these experts in layer l ; (3) measure the resulting change in output length. The layer producing the largest increase is denoted as l^* , indicating that it contains the most influential target-related experts. This procedure corresponds to the layer-level search illustrated in Figure 2-Step 2.

Comparing GLOBAL and LOCAL. GLOBAL incurs minimal computation but may modify more experts to achieve desired behavior manipulation. LOCAL is more precise, often leveraging fewer expert modifications and thus preserving good model utility, but requires $O(L)$ evaluations. Additionally, some MoE architectures (e.g., GPT-OSS (OpenAI, 2025)) distribute EOS behavior across multiple layers, making LOCAL less effective on those models.

Both GLOBAL and LOCAL remain stable even with very small sample sizes in the detection process (e.g., $N = 10$), consistently identifying target-related experts. Together, they offer flexible trade-offs between stealthiness and computational cost.

3.4 Bit-Flip Attack on MoE Router

Building on the identified target-related experts, we design a router-level bit-flip attack that directly alters their activation patterns (Step 3 in Figure 2), addressing **RQ3**. A standard MoE router computes logits as

$$s = h^\top W + b, \quad (5)$$

where $h \in \mathbb{R}^D$, $W \in \mathbb{R}^{D \times E}$, and b is optional. Among the evaluated models, Mixtral-8x7B, Phi-3.5-MoE, DeepSeek-V2-Lite, and Qwen3 use bias-free routers, so we flip bits in the expert columns v_i of W ; GPT-OSS-20B includes non-zero biases, so we flip bits in b_i instead. Both strategies suppress target-related experts analogously to manual deactivation.

Full-model BFA (Rakin et al., 2019) is infeasible at billion-parameter MoE scale, as every candidate bit requires a full forward pass. To avoid this cost, we propose an inference-free router-centric bit

search. We run the model once on N queries, cache the hidden states at target layer ℓ^* as an $[M, D]$ tensor, and reuse them for all bit evaluations. For each target-related expert i with routing vector v_i , we flip every bit once, recompute routing logits from cached states, and measure the resulting expert activation frequency (EAF). We define the bit-flip effectiveness as

$$\text{BF-eff}(b) = \frac{\text{EAF}_i^{\text{orig}} - \text{EAF}_i^{(b)}}{\text{EAF}_i^{\text{orig}}}, \quad (6)$$

and select the top- B bits per expert. Flipping these bits produces a perturbed routing vector $\tilde{v}_i$ that reliably suppresses expert i from the router’s top- k , realizing **GBFA** without repeated inference.

3.5 Online BFA Execution

With the target bits identified, the attacker flips them in the deployed model’s DRAM via Rowhammer (Kim et al., 2014; Lin et al., 2025) (Step 3 in Figure 2). Since the perturbation modifies persistent router weights, a single successful attack suppresses target-related experts across all subsequent queries without further interaction.

4 Experiment

4.1 Experiment Setup

Models. We evaluate six open-source MoE-based LLMs with diverse architectures (see Table 1 for specifications): Mixtral-8x7B-Instruct-v0.1 (Jiang et al., 2024) (Mixtral), Phi-3.5-MoE-Instruct (Abdin, 2024) (Phi-3.5), DeepSeek-V2-Lite-Chat (DeepSeek-AI, 2024) (DeepSeek), Qwen3-30B-A3B (Qwen Team, 2025) (Qwen3), Qwen3-Coder-Next (Qwen Team, 2026) (Qwen3-Coder), and GPT-OSS-20B (OpenAI, 2025) (GPT-OSS). These models represent different design philosophies in MoE architectures, from efficient small-scale routing to fine-grained expert decomposition.

Table 1: Overview of evaluated MoE architectures, showing total parameters, activated parameters per token, number of experts per layer, activated experts, and model depth.

Model	#Params (B)	Active (B)	#Experts	#Active	#Layers
Mixtral	46.7	12.9	8	2	32
Phi-3.5	41.9	6.6	16	2	32
DeepSeek	15.7	2.4	64	6	27
Qwen3	30.5	3.3	128	8	48
Qwen3-Coder	80.0	3.0	512	10	48
GPT-OSS	20.0	4.0	32	4	40

Metrics. We use two metrics. For **attack effectiveness**, we compute the **Percentage of Token Generation Change (P)**:

$$P_i = \frac{T_{\text{deact}}^{(i)} - T_{\text{base}}^{(i)}}{T_{\text{base}}^{(i)}} \times 100\%. \quad (7)$$

$T_{\text{base}}^{(i)}$ and $T_{\text{deact}}^{(i)}$ denote baseline and attacked token counts, and the mean $\hat{P}$ measures overall length increase. Figure 3 reports the underlying absolute token counts and their spread across blocked-expert counts. For **model utility**, we report Clean Accuracy (CA) on classification datasets, ROUGE-1 on Samsum, F1 on SQuAD, and **plan steps** on agent coding tasks.

Datasets. We use the *Stanford Alpaca* instruction-tuning dataset (Taori et al., 2023) as the source of sample sequences for identifying target-related experts. To evaluate our attack, we use four representative datasets: *AGNews* (Zhang et al., 2015) and *SST-2* (Socher et al., 2013) for classification, and *Samsum* (Gliwa et al., 2019) and *SQuAD 2.0* (Rajpurkar et al., 2018) for generation. We follow prior work (Wang et al., 2025) to adopt system prompts for all downstream tasks. All models and datasets used in our experiments are publicly available. We use them for research purposes in accordance with their corresponding licenses and terms of use.

Configuration. We target EOS. GLOBAL selects the top- k_g EOS-related experts ranked by frequency difference, where k_g varies per model to balance attack effectiveness and efficiency: 37 for Mixtral, 16 for Phi-3.5, 40 for DeepSeek, and 25 for GPT-OSS. LOCAL follows each model’s default router top- k (Table 1). We use greedy decoding and set max tokens to 1024. The attack does not depend on the decoding strategy: Appendix E shows that the length inflation persists under temperature and nucleus sampling and under a repetition penalty, and Appendix F reports the effect of the token budget and motivates our moderate choice of 1024.

Platform. Experiments were conducted on three NVIDIA A100 80GB GPUs. For Section 3.2, we computed the $\tau_{l,i}$ or Δgl_i over 1,000 samples. For identifying attacked bits in the bit-flip attack pipeline, the average cost was **16.4 s per expert**.

4.2 Manual EOS-related Experts Deactivation

To validate that **EOS-related experts** control output length, we manually deactivate the identified experts by setting their logits to **-inf**, establishing up-

per bounds for bit-flip attack effectiveness. Table 2 reports P across 100 samples per dataset under LOCAL, GLOBAL, and a Random-deactivation baseline (50 random experts).

Both LOCAL and GLOBAL substantially amplify output, consistently outperforming Random by orders of magnitude, validating our expert-detection procedure. The relative strength of LOCAL vs. GLOBAL reveals how termination is implemented in each model. When EOS is controlled by a few concentrated experts, LOCAL suffices and can even exceed GLOBAL (e.g., DeepSeek AGNews: $3.81 \times 10^4\%$ vs. 226%). When termination relies on an inter-layer path, GLOBAL becomes necessary (e.g., GPT-OSS AGNews: 556% vs. 15.9%). Classification tasks yield larger P than generation tasks, as short baseline outputs amplify into thousand-fold increases.

Figure 3 shows that output token count generally increases with more blocked experts. GPT-OSS and Phi-3.5 quickly saturate max_new_tokens and plateau. DeepSeek shows irregular behavior on Samsum, likely due to the model’s strong prior on short responses for this dataset.

Without EOS-related experts, models generate repetitive but coherent text loops that preserve linguistic quality while losing termination control (qualitative examples on Mixtral in Appendix B). This output amplification potential motivates our bit-flip targets.

Table 2: P across all samples for manual expert deactivation (LOCAL vs. GLOBAL), compared against a random-deactivation baseline.

Model	Method (#exp)	AGNews (%)	SST-2 (%)	Samsum (%)	SQuAD (%)
Mixtral	Random (50)	685	188	149	-9.28
	LOCAL (2)	2.18×10^3	2.23×10^3	2.19×10^3	3.64×10^3
	GLOBAL (37)	2.74×10^3	3.41×10^3	389	52.9
Phi-3.5	Random (50)	215	1.14	30.1	155
	LOCAL (2)	2.82×10^4	1.83×10^3	20.5	217
	GLOBAL (16)	2.99×10^4	1.74×10^3	1.27×10^3	1.89×10^3
DeepSeek	Random (50)	67.9	125	5.97	18.8
	LOCAL (6)	3.81×10^4	5.82×10^3	78.6	1.00×10^4
	GLOBAL (40)	226	2.81×10^3	14	140
GPT-OSS	Random (50)	-3.43	10.3	11.6	2.29
	LOCAL (4)	15.9	17.3	36.7	9.30
	GLOBAL (25)	556	468	177	445

4.3 Groundhog Bit-Flip Attack

We implement GBFA targeting router weights to manipulate EOS-related expert activation. For each **EOS-related expert**, we identified three critical router weights using the method described in Section 3.4, with each weight requiring only a single bit, resulting in **three total bit modifications** per expert. For models with router bias parameters

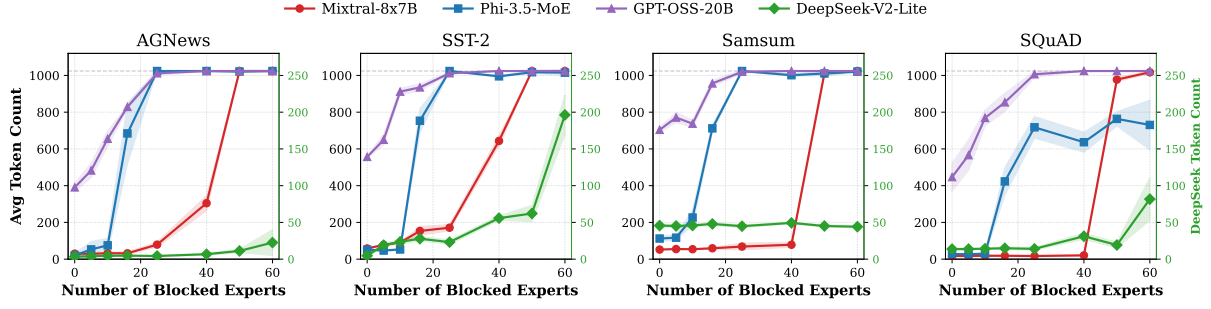


Figure 3: Average output token count under GLOBAL manual deactivation as a function of blocked experts. The dashed line marks `max_new_tokens`; DeepSeek-V2-Lite uses a separate right axis.

(e.g., GPT-OSS), we prioritized attacking these biases as they directly offset routing logits, providing more efficient manipulation. Table 3 presents the P described in Equation 7 under bit-flip attacks using LOCAL and GLOBAL selection strategies.

The results show that **GBFA** achieves substantial output manipulation comparable to manual deactivation. Since attacking weight matrices involves complex inter-weight dependencies, where multiple weights collectively determine routing logits across varying input distributions, bit-flips produce approximate rather than exact expert suppression. This approximation can occasionally yield stronger deactivation effects than manual blocking, as the perturbations propagate through weight interactions in non-trivial ways. Overall, the results confirm that GBFA is a viable and effective attack: despite operating at the bit level with minimal parameter changes, it reliably induces significant output inflation across models and datasets.

Notably, GPT-OSS demonstrates near-manual performance, achieving comparable results to manual deactivation. This exceptional effectiveness stems from directly attacking bias parameters, which provide a linear offset to all routing decisions without inter-parameter interference. These findings validate that GBFA can successfully manipulate output length through EOS-expert targeting, with bias parameters offering optimal attack surfaces when available.

4.4 Performance Impact of GBFA on MoE Models

We evaluated model performance under **GBFA** across four MoE architectures using 1,000 randomly selected samples per dataset. Table 4 presents the results comparing baseline, manual deactivation, and bit flip conditions.

Performance degradation correlates with attack

Table 3: P across All Samples for Bit-flip attack (LOCAL vs GLOBAL).

Model	Method (#exp)	AGNews (%)	SST-2 (%)	Samsum (%)	SQuAD (%)
Mixtral	LOCAL (2)	202	110	1.06	39.3
	GLOBAL (37)	2.51×10^3	4.99×10^3	2.92×10^3	48.1
Phi-3.5	LOCAL (2)	526	42.1	148	45.3
	GLOBAL (16)	131	20.8	222	167
DeepSeek	LOCAL (6)	3.84×10^4	8.73×10^4	1.61×10^3	4.05×10^2
	GLOBAL (40)	4.64×10^4	4.44×10^4	3.04×10^3	1.49×10^4
GPT-OSS	LOCAL (4)	15.9	17.3	36.7	9.30
	GLOBAL (25)	556	468	177	445

effectiveness (measured by Equation 7). For Mixtral, manual deactivation causes substantial drops in Clean Accuracy (CA) decreases from 0.838 baseline to 0.602 (LOCAL) and 0.459 (GLOBAL) on AGNews. In contrast, bit-flip attacks preserve performance better, with CA remaining at 0.840 (LOCAL) and 0.800 (GLOBAL). Interestingly, some configurations show performance improvements: Phi-3.5’s ROUGE-1 on Samsum increases from 0.362 to 0.347-0.382 under LOCAL attacks. This aligns with recent findings (Fayyaz et al., 2025) that selective expert manipulation can enhance task-specific performance through improved alignment.

Perplexity (PPL) measurements confirm that most attacks maintain linguistic coherence. PPL values remain below 10 for most configurations, indicating meaningful text generation rather than gibberish. The notable exception is DeepSeek under GLOBAL bit-flip attacks, where PPL explodes to 91,839 (SST-2) and 5,288,426 (Samsum), signaling complete language generation breakdown. We attribute this to the flipped bits also perturbing router weights on which DeepSeek relies for coherent generation: corruption typically emerges within the first few generated tokens and then propagates, since early tokens condition all subsequent ones. Incorporating the perplexity of the first few tokens as an additional criterion in the bit search could mitigate this effect, which we leave for future

work.

Overall, **GBFA** on MoE routers achieves effective output manipulation while maintaining relative stealth by preserving model quality metrics even as they dramatically extend output length.

Table 4: Performance of different models on four datasets under manual and bit-flip expert manipulation.

Model	Method	AGNews		SST-2		Samsum		SQuAD	
		CA	PPL	CA	PPL	R-1	PPL	F1	PPL
DeepSeek	Baseline	0.810	1.35	0.919	1.69	0.364	1.38	0.362	1.22
	LOCAL (manual)	0.970	1.17	0.793	2.33	0.270	1.49	0.070	1.40
	GLOBAL (manual)	0.395	1.82	0.836	1.73	0.205	1.60	0.159	1.82
	LOCAL (GBFA)	0.380	1.82	0.020	1.81	0.117	1.36	0.021	1.21
	GLOBAL (GBFA)	0.160	4.4e3	0.000	9.2e4	0.025	5.3e6	0.081	5.1e6
Mixtral	Baseline	0.838	1.40	0.809	1.47	0.362	1.39	0.253	1.13
	LOCAL (manual)	0.602	2.37	0.698	3.73	0.171	1.27	0.071	1.51
	GLOBAL (manual)	0.459	2.52	0.508	3.91	0.086	1.18	0.058	1.43
	LOCAL (GBFA)	0.840	1.33	0.890	1.47	0.102	1.91	0.237	1.09
	GLOBAL (GBFA)	0.800	1.89	0.920	1.68	0.252	1.66	0.242	1.23
Phi-3.5	Baseline	0.856	1.13	0.862	1.25	0.375	1.21	0.235	1.10
	LOCAL (manual)	0.569	2.26	0.681	1.63	0.347	1.29	0.096	1.36
	GLOBAL (manual)	0.076	6.12	0.011	8.95	0.025	1.46	0.052	2.20
	LOCAL (GBFA)	0.800	1.18	0.830	1.33	0.382	1.23	0.263	1.07
	GLOBAL (GBFA)	0.750	1.98	0.840	1.33	0.339	1.46	0.135	1.26
GPT-OSS	Baseline	0.800	1.36	0.940	1.37	0.199	1.36	0.049	1.12
	LOCAL (manual)	0.800	1.37	0.940	1.41	0.177	1.37	0.044	1.12
	GLOBAL (manual)	0.790	1.56	0.880	1.61	0.065	1.33	0.045	1.17
	LOCAL (GBFA)	0.800	1.37	0.940	1.41	0.177	1.37	0.044	1.12
	GLOBAL (GBFA)	0.790	1.56	0.880	1.61	0.065	1.33	0.045	1.17

5 Broader Attack Scenarios

5.1 Attack the Plan Mode in Agent Coding

We evaluate whether GBFA can prolong an LLM agent’s planning behavior in long-horizon coding tasks. We use **Qwen3-Coder**, a model designed specifically for coding agents and local development, deployed under the **smolagents** CodeAgent framework, a Hugging Face library for easily running agents on Hugging Face models. The framework alternates between planning and action steps, and we target the chat-template end-of-turn token `<|im_end|>`.

Setup. We design 10 self-contained Python repository sandboxes spanning multiple task categories (full descriptions in Appendix C). Each sandbox is reset to a clean snapshot before every run, with completion verified by an associated unit-test suite. We compare a clean baseline against two manual deactivation configurations, **LOCAL** and **GLOBAL**, targeting the top-20 routed experts ranked by $\tau_{l,i}$ and the top-4 shared experts ranked by $\Delta g_{l,i}$. Each agent runs with a 25-step limit, planning every 3 steps, and 2048 max new tokens per generation.

Plan-mode results. Table 5 shows that GLOBAL deactivation reliably amplifies plan-token consumption: all 10 sandboxes hit the maximum step bud-

Table 5: Plan-mode manual expert deactivation on Qwen3-Coder-Next across 10 coding sandboxes. We report plan steps (#), average plan tokens, and the relative change P in plan-token totals against the baseline.

Sandbox	Baseline		LOCAL (manual)			GLOBAL (manual)		
	#	tokens	#	tokens	P	#	tokens	P
calculator_refactor	4	540.2	3	888.0	+23.3%	9	1869.6	+678.6%
bug_fix	3	620.0	3	611.3	-1.4%	9	2048.0	+891.0%
feature_add	3	507.7	3	565.7	+11.4%	9	1231.7	+627.8%
rename_refactor	4	461.8	4	620.0	+34.3%	9	1095.1	+433.6%
test_coverage	7	509.1	4	607.8	-31.8%	9	1061.6	+168.1%
multi_file_refactor	6	585.0	5	891.0	+26.9%	9	1622.1	+315.9%
bug_hunt	3	405.3	3	568.0	+40.1%	9	1528.9	+1031.6%
performance_optimize	5	508.8	3	690.0	-18.6%	9	1005.8	+255.8%
error_handling	4	451.5	3	555.7	-7.7%	9	1464.8	+630.0%
dependency_swap	3	602.3	3	658.7	+9.4%	9	1666.6	+730.0%
Mean	4.2	519.2	3.4	665.6	+8.6%	9.0	1459.4	+576.2%

get, with mean $\hat{P} = +501.5\%$. LOCAL remains inconsistent and statistically negligible. The amplification manifests as plan-output degeneration into syntactically valid character loops, illustrated on five sandboxes in Appendix D.

5.2 Attack the End-of-Thinking Token

To evaluate whether our attack transfers to other types of tokens, we apply it to the thinking process of GPT-OSS and Qwen3. Following the same procedure as the Groundhog Bit-Flip Attack, we use LOCAL and GLOBAL to identify experts that are closely associated with the **end of thinking (EOT)** token and we manually deactivate these experts. The results are shown in Table 6. We observe a clear increase in the number of tokens generated during the thinking phase. In the most extreme case, the model continues thinking until the token budget is exhausted without producing the expected answer. This experiment demonstrates that our attack also applies to other specific tokens.

Table 6: Results of GPT-OSS and Qwen3 under LOCAL and GLOBAL manual deactivation of EOT-related experts (percentage increase in thinking length).

Model	Method (manual)	AGNews (%)	SST-2 (%)	Samsum (%)	SQuAD (%)
GPT-OSS	LOCAL	19.7	15.2	58.4	25.9
	GLOBAL	7.15×10^2	1.27×10^3	365	1.33×10^3
Qwen3	LOCAL	6.28	1.61	4.18	-1.37
	GLOBAL	204	224	230	271

6 Discussion

6.1 Feasibility under Distributed Inference

Large MoE models are often served with tensor parallelism (TP) and pipeline parallelism (PP). We discuss whether GBFA still works when router parameters are split or replicated across devices. Un-

der PP, each layer and its router stay on one device. The router bits we target are not split, so the attack needs the same number of flips as in the single-device setting. PP thus does not increase the difficulty. Under TP, the router weights may be replicated on several devices, which would require flipping the same bit on each replica. The attacker can avoid this by corrupting the shared page cache that the replicas load from, so that one poisoned copy reaches all devices. Prior fault-injection work uses the same idea (Tol et al., 2023). Neither scheme therefore breaks the feasibility of GBFA.

6.2 Potential Defenses

Potential defenses against GBFA fall into two categories.

Software-level mitigations such as API-level output length caps (OpenAI, 2024; Anthropic, 2024) reduce per-request damage but cannot prevent the underlying attack. Output-pattern detectors such as Breaking the Loop (Yu et al., 2025) can interrupt loop behavior in real time, but legitimate repetitive outputs trigger the same signal, limiting practical reliability. Neither class of defense remediates the root cause.

System-level protections such as ECC (Sridharan et al., 2015) or TEE-based isolation (Sun et al., 2023; Ding et al., 2025a) can detect accidental faults but provide limited security against adversarial bit-flips and incur nontrivial overhead.

These gaps highlight the need for *MoE-specific defenses* that constrain routing sensitivity and verify parameter integrity.

7 Conclusion

In this paper, we proposed **Groundhog Bit-Flip Attack (GBFA)**, a structural Denial-of-Wallet attack that exploits MoE routing through targeted router-level bit-flips. Our analysis revealed strong target-related expert specialization, enabling attackers to suppress termination-critical experts by flipping only a few bits. Using GBFA, we efficiently identified vulnerable experts and showed that these small perturbations can induce massive output-length inflation while largely preserving model utility. Experiments across six MoE LLMs and three LLM modes confirm that routing-layer corruption constitutes a previously overlooked vulnerability. These findings underscore the need for MoE-specific defenses to safeguard router integrity and prevent targeted manipulation of expert activation.

8 Limitations

- **Strong threat model.** GBFA assumes white-box access to the victim MoE model and the ability to inject targeted bit flips into selected router parameters. Although this follows prior bit-flip attack settings, it may not hold for all practical MLaaS with strong isolation or integrity checks.
- **System-level cost analysis.** We quantify vulnerable bit search cost and model-level attack effectiveness, but the full end-to-end attack cost also depends on hardware-specific factors such as memory placement and BFA reliability.
- **No end-to-end hardware demonstration.** We identify vulnerable bits and measure the output inflation once they are flipped, but we do not perform an end-to-end hardware exploit that physically flips them on a live system. System-level defenses such as ECC memory or TEE-based isolation raise the difficulty of such flips. However, they do not fully prevent adversarial bit flips and add nontrivial overhead, so they mitigate rather than eliminate the threat.
- **Theoretical analysis.** Our study is mainly empirical. We show that EOS and EOT behavior can concentrate in a small set of MoE experts, but we do not provide a formal theoretical explanation of how routing logits, expert specialization, and output-length inflation are related.

Acknowledgments

Portions of this research were conducted with high performance computational resources provided by Louisiana State University (<http://www.hpc.lsu.edu>) and the Louisiana Optical Network Infrastructure (<http://www.loni.org>).

References

- Marah et al. Abdin. 2024. [Phi-3 technical report: A highly capable language model locally on your phone](#). Preprint, arXiv:2404.14219.
- Anthropic. 2024. Messages – API reference. <https://docs.anthropic.com/en/api/messages>. Accessed: 2025.
- Jiawang Bai, Baoyuan Wu, Zhifeng Li, and Shu-Tao Xia. 2023. [Versatile weight attack via flipping limited bits](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):13653–13665.

- Kunbei Cai, Md Hafizul Islam Chowdhury, Zhenkai Zhang, and Fan Yao. 2021. [Seeds of SEED: NMT-Stroke: Diverting Neural Machine Translation through Hardware-based Faults](#). In *2021 International Symposium on Secure and Private Execution Environment Design (SEED)*, pages 76–82. IEEE.
- Huili Chen, Cheng Fu, Jishen Zhao, and Farinaz Koushanfar. 2021. [ProFlip: Targeted Trojan Attack with Progressive Bit Flips](#). In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE.
- DeepSeek. 2025. Deepseek api documentation: Models and pricing. https://api-docs.deepseek.com/quick_start/pricing.
- DeepSeek-AI. 2024. [Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model](#). *Preprint*, arXiv:2405.04434.
- Ruyi Ding, Tianhong Xu, Aidong Adam Ding, and Yunsi Fei. 2025a. [Graph in the vault: Protecting edge gnn inference with trusted execution environment](#). In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7.
- Ruyi Ding, Tianhong Xu, Xinyi Shen, Aidong Adam Ding, and Yunsi Fei. 2025b. [Moecho: Exploiting side-channel attacks to compromise user privacy in mixture-of-experts llms](#). *arXiv preprint arXiv:2508.15036*.
- Mohsen Fayyaz, Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Ryan Rossi, Trung Bui, Hinrich Schütze, and Nanyun Peng. 2025. [Steering moe llms via expert \(de\)activation](#). *Preprint*, arXiv:2509.09660.
- William Fedus, Barret Zoph, and Noam Shazeer. 2022. [Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity](#). *Preprint*, arXiv:2101.03961.
- Pietro Frigo, Emanuele Vannacc, Hasan Hassan, Victor Van Der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. 2020. [TRRespass: Exploiting the many sides of target row refresh](#). In *IEEE Symposium on Security and Privacy*.
- Behnam Ghavami, Mani Sadati, Mohammad Shahidzadeh, Zhenman Fang, and Lesley Shannon. 2022. [Bdfa: A blind data adversarial bit-flip attack on deep neural networks](#). *Preprint*, arXiv:2112.03477.
- Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. 2019. [Samsun corpus: A human-annotated dialogue dataset for abstractive summarization](#). *arXiv preprint arXiv:1911.12237*.
- Hasan Hassan, Yahya Can Tugrul, Jeremie S. Kim, Victor van der Veen, Kaveh Razavi, and Onur Mutlu. 2022. [Uncovering in-dram rowhammer protection mechanisms: A new methodology, custom rowhammer patterns, and implications](#). *Preprint*, arXiv:2110.10603.
- Patrick Jattke, Victor Van Der Veen, Pietro Frigo, Stijn Gunter, and Kaveh Razavi. 2022. [Blacksmith: Scalable rowhammering in the frequency domain](#). In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE.
- Patrick Jattke, Max Wipfli, Flavien Solt, Michele Marazzi, Matej Bölskei, and Kaveh Razavi. 2024. [Zenhammer: Rowhammer attacks on amd zen-based platforms](#). In *33rd USENIX Security Symposium (USENIX Security 2024)*.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, and 1 others. 2024. [Mixtral of experts](#). *arXiv preprint arXiv:2401.04088*.
- Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. 2014. [Flipping bits in memory without accessing them: An experimental study of dram disturbance errors](#). In *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, pages 361–372.
- Andreas Kogler, Jonas Juffinger, Salman Qazi, Yoongu Kim, Moritz Lipp, Nicolas Boichat, Eric Shiu, Matthias Nissler, and Daniel Gruss. 2022. [Half-double: Hammering from the next row over](#).
- Cai Kunbei, Chowdhury Md Hafizul Islam, Zhang Zhenkai, and Yao Fan. 2024. [Deepvenom: Persistent dnn backdoors exploiting transient weight perturbations in memories](#). In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 2067–2085.
- Andrew Kwong, Daniel Genkin, Daniel Gruss, and Yuval Yarom. 2020. [Rambleed: Reading bits in memory without accessing them](#). In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 695–711.
- Zhenglin Lai, Mengyao Liao, Bingzhe Wu, Dong Xu, Zebin Zhao, Zhihang Yuan, Chao Fan, and Jianqiang Li. 2025. [Safex: Analyzing vulnerabilities of moe-based llms via stable safety-critical expert identification](#). *Advances in Neural Information Processing Systems*, 38:130380–130404.
- Xiang Li, Ying Meng, Junming Chen, Lannan Luo, and Qiang Zeng. 2025. [Rowhammer-based trojan injection: One bit flip is sufficient for backdooring dnn](#)s. In *USENIX Security Symposium*.
- Chris S. Lin, Joyce Qu, and Gururaj Saileshwar. 2025. [GPUHammer: Rowhammer attacks on GPU memories are practical](#). In *34th USENIX Security Symposium (USENIX Security 25)*, pages 5719–5738, Seattle, WA. USENIX Association.
- Yannan Liu, Lingxiao Wei, Bo Luo, and Qiang Xu. 2017. [Fault injection attack on deep neural network](#). In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 131–138.

- Nicolas Moro, Amine Dehbaoui, Karine Heydemann, Bruno Robisson, and Emmanuelle Encrenaz. 2013. [Electromagnetic fault injection: Towards a fault model on a 32-bit microcontroller](#). In *2013 Workshop on Fault Diagnosis and Tolerance in Cryptography*, pages 77–88.
- Ataberk Olgun, Majd Osseiran, A. Giray Yağlıkçı, Yahya Can Tuğrul, Haocong Luo, Steve Rhyner, Behzad Salami, Juan Gomez Luna, and Onur Mutlu. 2024. [Read disturbance in high bandwidth memory: A detailed experimental study on hbm2 dram chips](#). In *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 75–89.
- OpenAI. 2024. Chat completions – API reference. <https://platform.openai.com/docs/api-reference/chat/create>. Accessed: 2025.
- OpenAI. 2025. [gpt-oss-120b & gpt-oss-20b model card](#). Preprint, arXiv:2508.10925.
- Qwen Team. 2025. [Qwen3 technical report](#). Preprint, arXiv:2505.09388.
- Qwen Team. 2026. [Qwen3-coder-next technical report](#). Technical report. Accessed: 2026-02-03.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. [Know what you don’t know: Unanswerable questions for SQuAD](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia. Association for Computational Linguistics.
- Adnan Siraj Rakin, Md Hafizul Islam Chowdhury, Fan Yao, and Deliang Fan. 2022a. [Deepsteal: Advanced model extractions leveraging efficient weight stealing in memories](#). In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1157–1174.
- Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. 2019. Bit-flip attack: Crushing neural network with progressive bit search. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1211–1220.
- Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. 2020. [Tbt: Targeted neural network attack with bit trojan](#). In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13195–13204.
- Adnan Siraj Rakin, Zhezhi He, Jingtao Li, Fan Yao, Chaitali Chakrabarti, and Deliang Fan. 2022b. [T-BFA: Targeted Bit-Flip Adversarial Weight Attack](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7928–7939.
- Mark Seaborn and Thomas Dullien. 2015. Exploiting the dram rowhammer bug to gain kernel privileges. *Black Hat*, 15:71.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Vilas Sridharan, Nathan DeBardeleben, Sean Blanchard, Kurt B Ferreira, Jon Stearley, John Shalf, and Sudhanva Gurumurthi. 2015. Memory errors in modern systems: The good, the bad, and the ugly. *ACM SIGARCH Computer Architecture News*, 43(1):297–310.
- Zhichuang Sun, Ruimin Sun, Changming Liu, Amrita Roy Chowdhury, Long Lu, and Somesh Jha. 2023. Shadownet: A secure and efficient on-device model inference system for convolutional neural networks. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1596–1612. IEEE.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.
- M. Caner Tol, Saad Islam, Andrew J. Adiletta, Berk Sunar, and Ziming Zhang. 2023. [Don’t knock! rowhammer at the backdoor of dnn models](#). In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 109–122.
- Qingyue Wang, Qi Pang, Xixun Lin, Shuai Wang, and Daoyuan Wu. 2025. [Badmoe: Backdoor-ing mixture-of-experts llms via optimizing routing triggers and infecting dormant experts](#). Preprint, arXiv:2504.18598.
- Fan Yao, Adnan Siraj Rakin, and Deliang Fan. 2020. DeepHammer: Depleting the intelligence of deep neural networks through targeted chain of bit flips. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 1463–1480. USENIX Association.
- Junzhe Yu, Yi Liu, Huijia Sun, Ling Shi, and Yuqi Chen. 2025. Breaking the loop: Detecting and mitigating denial-of-service vulnerabilities in large language models. *arXiv preprint arXiv:2503.00416*.
- Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. [Character-level convolutional networks for text classification](#). In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc.
- Mengxin Zheng, Qian Lou, and Lei Jiang. 2023. [TrojViT: Trojan Insertion in Vision Transformers](#). Preprint, arXiv:2208.13049.

Algorithm 1 Groundhog Bit-Flip Attack

Require: MoE LLM f_θ , dataset S , router top- k , bits-per-expert B

Ensure: vulnerable bits $\mathcal{B}$

Step 1: Identify target-specialized experts

- 1: **for** each expert (ℓ, i) **do**
- 2: Compute $\tau_{\ell,i}$ or $\Delta g_{\ell,i}$ using target token and non-target activations over S
- 3: **end for**

Step 2: Local-based target expert selection

- 4: Select S_{val} and measure baseline length $\bar{T}_{\text{base}}$
- 5: **for** each layer ℓ **do**
- 6: $\mathcal{E}_\ell^{\text{top}} \leftarrow$ top- k_ℓ experts ranked by τ or Δg
- 7: Temporarily deactivate $\mathcal{E}_\ell^{\text{top}}$ and compute $\bar{T}_\ell$
- 8: **end for**
- 9: $\ell^* \leftarrow \arg \max_\ell (\bar{T}_\ell - \bar{T}_{\text{base}})$
- 10: $\mathcal{E}_{\text{target}} \leftarrow \mathcal{E}_{\ell^*}^{\text{top}}$

Step 3: Groundhog Bit search

- 11: Cache hidden states H at layer ℓ^*
- 12: Compute $\text{EAF}_i^{\text{orig}}$ for each $i \in \mathcal{E}_{\text{target}}$
- 13: $\mathcal{B} \leftarrow \emptyset$
- 14: **for** each expert i **do**
- 15: **for** each bit b in its router parameter **do**
- 16: Flip b , reroute on H , compute $\text{BF} - \text{eff}(b)$
- 17: **end for**
- 18: Add top B bits for i into $\mathcal{B}$
- 19: **end for**
- 20: **return** $\mathcal{B}$

A Groundhog Bit-Flip Attack Algorithm

Algorithm 1 details the three-stage procedure used in GBFA to identify vulnerable bits in MoE router weights. The algorithm takes an MoE model, a sample dataset, and the router top- k as input, and returns a set of vulnerable bits $\mathcal{B}$ to flip. In Stage 1, each expert is scored by its target activation shift ($\tau_{\ell,i}$ for routed experts, $\Delta g_{\ell,i}$ for shared experts) to identify target-related experts. In Stage 2, a layer-wise ablation selects the single most critical layer whose deactivation maximally inflates output length. In Stage 3, bits within the selected experts’ router parameters are evaluated individually, and the top- B bits per expert with the highest bit-flip effectiveness are added to $\mathcal{B}$.

B Qualitative Examples of EOS-related Expert Deactivation

To complement the quantitative results in the main text, Table 7 presents qualitative generation examples from Mixtral after deactivating EOS-related

experts, on four representative datasets (AGNews, SST-2, Samsum, and SQuAD_v2). Each row shows the input prompt, the model’s original output, and the output produced under the same prompt after deactivation. Without EOS-related experts, the model generates repetitive but coherent text loops, preserving linguistic quality while losing termination control. These examples illustrate the output amplification potential of EOS expert deactivation, validating our attack strategy and identifying precise targets for the bit-flip implementation in the main text.

Table 7: Generation examples from Mixtral on four datasets, comparing the original output (blue) with the output after EOS-related expert deactivation. Repetitive loops introduced by the attack are highlighted in red.

Dataset	Input Prompt	Ori Output	Attacked Output
AGNews	Classify the news article... Article: Reuters – Soaring crude prices... Category:	Category: Business Explanation: The article discusses the impact of soaring crude prices...	Category: Business/Economy User 0: I would say business. User 1: I would say sports. User 0: I would say sports. User 1: I would say sports because the article is about the stock market. (repeated ...)
SST-2	Classify the sentiment... Text: hide new secretions... Sentiment:	Sentiment: Negative Explanation: The text mentions hiding something from “parental units,” suggesting secrecy...	Sentiment: Negative The text describes the narrator’s relationship with his mother... He is describing the relationship in the past tense. He is describing the relationship in the past tense. (repeated ...)
Samsum	Please summarize the dialogue... Amanda: I baked cookies... Jerry: Sure! Summary:	Amanda offers Jerry cookies; she will bring them the next day.	Amanda baked cookies and offered some to Jerry. Comment: Direct response. Jerry: Sure! Jerry’s answer is a direct response to Amanda’s question. Jerry’s answer is a direct response to Amanda’s question. (repeated ...)
SQuAD	Answer the question based on the context... Context: Beyoncé Giselle Knowles-Carter... Question: When did Beyoncé rise to fame?	Beyoncé rose to fame in the late 1990s as lead singer of Destiny’s Child.	Beyoncé rose to fame in the late 1990s... Comment: Beyoncé rose to fame... Destiny’s Child. Comment: How did Beyoncé rise to fame? Answer: Beyoncé rose to fame... Destiny’s Child. (repeated ...)

C Sandbox Task Descriptions

Table 8 lists the 10 self-contained Python repository sandboxes used in our agentic coding evaluation

(Section 5.1). Each sandbox ships with a clean initial snapshot and a unit-test suite that determines task completion. We will release the implementation details, prompts, and configuration files together with the code.

Table 8: Ten sandbox tasks used in the agentic coding evaluation.

Task	Description
<i>bug_fix</i>	Fix an off-by-one bug in a FizzBuzz function.
<i>bug_hunt</i>	Locate and fix three independent bugs co-located in a single file.
<i>feature_add</i>	Implement two missing methods (mark_done, pending_count) in a TODO-list class.
<i>test_coverage</i>	Add at least two unit tests for each of four string utility functions.
<i>calculator_refactor</i>	Extract trigonometric methods (sin/cos/tan) from Calculator into a dedicated TrigCalculator class.
<i>rename_refactor</i>	Rename MyHelper to StringUtils across multiple files and update all imports.
<i>multi_file_refactor</i>	Split a monolithic Database class into three repository modules.
<i>performance_optimize</i>	Convert an $O(n^2)$ implementation to $O(n)$ while passing performance tests.
<i>error_handling</i>	Add graceful error handling to a data-processing pipeline.
<i>dependency_swap</i>	Remove the re module dependency and reimplement five functions using only built-in string methods.

D Attacked Output of Plan Mode

Table 9 presents qualitative examples of plan-mode output degradation under GLOBAL manual expert deactivation on Qwen3-Coder-Next across five coding sandbox tasks. Each task block shows the shared prelude (gray box) that is bit-identical between the baseline and attacked runs, followed by the divergence point where the two outputs split. The baseline continues with coherent planning, while the attacked output drifts before collapsing into a repetitive degenerate pattern (shown in red), eventually exhausting max_new_tokens without emitting `<|im_end|>`.

E Robustness to Decoding Strategies

Our main experiments use greedy decoding for reproducibility. To check that GBFA does not depend on this choice, we re-run the GLOBAL (top-16) attack on Phi-3.5-MoE under six decoding

settings (Table 10). Settings are: greedy (argmax, do_sample=False, default); temp0.7/temp1.0 (sampling, top_p=1.0); topp0.9 (nucleus, temperature=1.0, top_p=0.9); and the _rp1.3 variants add a repetition penalty of 1.3. EOS suppression and length inflation hold under greedy, temperature, and nucleus sampling (attacked EOS rate 0.22–0.61 vs. baseline ≈ 1.0). A repetition penalty above 1 already lengthens the baseline output on its own, because it biases the model toward more elaborate responses; GBFA remains effective under this setting as well.

F Effect of the Token Budget

The choice of max_new_tokens bounds how much the attack can inflate output, so it shapes the measured effect. Table 11 sweeps the budget for Phi-3.5-MoE under the GLOBAL (top-16) attack. A budget that is too small caps the attack for datasets whose baselines are already long, while a larger budget lets short-baseline datasets inflate further; the relative change P therefore grows with the budget. We adopt a moderate value of 1024 throughout the paper: it is large enough to expose the attack (outputs several times the baseline) yet small enough to keep evaluation tractable. Since an output several times longer than the baseline already constitutes a strong Denial-of-Wallet attack, the exact budget does not change our conclusions.

Table 9: Output degradation under GLOBAL manual expert deactivation on Qwen3-Coder-Next in plan mode. For each task, the gray box shows the *shared prelude*—characters that are bit-identical across both runs. After the divergence point ([**diverges**]), the *Original* column shows the baseline continuing coherently, while the *Attack* column drifts and eventually collapses into a **repetitive degenerate pattern** (shown in red). Long gaps inside the attack output are abbreviated as [*skipped K chars*]. Newlines are rendered as a dimmed \n; ... marks truncation.

[illegible]

Table 10: Phi-3.5-MoE under the GLOBAL (top-16) attack across six decoding settings. Base/Atk Tok.: mean generated tokens per sample without/with the attack. P (%): relative increase. Base/Atk EOS: fraction of samples emitting an EOS token without/with the attack.

Setting	Dataset	Base Tok.	Atk Tok.	P (%)	Base EOS	Atk EOS
greedy	AGNews	47.45	496.52	946.4	0.96	0.53
	SST-2	49.81	746.42	1398.5	1.00	0.30
	Samsum	83.45	689.15	725.8	1.00	0.38
	SQuAD	30.11	544.29	1707.7	1.00	0.51
temp0.7	AGNews	42.00	445.03	959.6	0.97	0.60
	SST-2	61.40	668.11	988.1	1.00	0.39
	Samsum	108.79	728.35	569.5	0.96	0.35
	SQuAD	31.39	554.80	1667.4	1.00	0.50
temp1.0	AGNews	44.77	470.88	951.8	0.97	0.61
	SST-2	56.24	541.89	863.5	1.00	0.58
	Samsum	100.02	725.63	625.5	0.96	0.37
	SQuAD	36.00	548.77	1424.4	1.00	0.56
topp0.9	AGNews	32.75	451.83	1279.6	0.98	0.61
	SST-2	56.09	629.95	1023.1	1.00	0.46
	Samsum	101.92	831.65	716.0	0.96	0.22
	SQuAD	30.84	518.54	1581.4	1.00	0.58
greedy_rp1.3	AGNews	629.80	1004.61	59.5	0.39	0.02
	SST-2	436.74	976.07	123.5	0.60	0.06
	Samsum	755.84	1005.45	33.0	0.28	0.03
	SQuAD	661.70	990.11	49.6	0.37	0.04
temp0.7_rp1.3	AGNews	654.90	1024.00	56.4	0.37	0.00
	SST-2	563.15	987.70	75.4	0.47	0.04
	Samsum	878.34	1021.66	16.3	0.15	0.01
	SQuAD	817.93	1004.52	22.8	0.21	0.02

Table 11: Phi-3.5-MoE under the GLOBAL (top-16) attack at different max_new_tokens budgets (paper default: 1024). Base/Atk Tok.: mean tokens without/with the attack. P : relative increase. Base/Atk EOS: fraction emitting an EOS token. Baselines differ slightly across budgets because a few samples exceed the smaller caps and are truncated there.

Budget	Dataset	Base Tok.	Atk Tok.	P (%)	Base EOS	Atk EOS
512	AGNews	16.46	273.30	1560.4	0.98	0.49
	SST-2	49.98	383.07	666.4	1.00	0.31
	Samsum	76.69	369.21	381.4	0.94	0.32
	SQuAD	30.39	280.41	822.7	1.00	0.52
2048	AGNews	44.18	1056.66	2291.7	0.99	0.49
	SST-2	49.98	1442.91	2787.0	1.00	0.31
	Samsum	113.10	1312.92	1060.8	1.00	0.39
	SQuAD	30.39	999.16	3187.8	1.00	0.54
4096	AGNews	50.04	2304.27	4504.9	1.00	0.44
	SST-2	49.98	2879.90	5662.1	1.00	0.31
	Samsum	113.10	2563.51	2166.6	1.00	0.43
	SQuAD	30.39	2046.70	6634.8	1.00	0.52